

Plan alimentaire — expérience complète + code utilisé

Ce document explique, étape par étape, **tout ce qui se passe** quand un plan alimentaire en PDF est importé, jusqu'à l'utilisation par la cliente, y compris **comment les équivalences d'aliments sont prises en compte**. Le **code réel** utilisé est donné à la fin de chaque partie.

1. L'import du PDF (côté coach)

Tout part du tableau de bord coach (espace admin). Pour une cliente donnée, le coach charge son **plan alimentaire en PDF**. Le fichier est converti en texte encodé (Base64) puis envoyé au serveur pour analyse.

Code — endpoint d'analyse (`server/admin-routes.ts`)

```
POST /api/admin/clients/:id/analyze-pdf
Corps : { "pdfBase64": "...", "mimeType": "application/pdf" }
```

2. L'analyse automatique par l'IA

Le serveur lit le PDF et **extraît tout automatiquement** (aucune saisie manuelle). Le résultat est renvoyé sous une forme structurée stricte (JSON).

Code — forme des données extraites

```
{
  "allergies": ["string"],
  "intolerances": ["string"],
  "restrictions": ["string"],
  "rules": [{ "rule": "string", "importance": "high/medium/low" }],
  "calorie_recommendation": 0,
  "protein_recommendation": 0,
  "notes": "string",
  "pathologies": ["string"],
  "supplements": ["string"]
}
```

À cela s'ajoutent les **recettes/repas** et les **tables d'équivalences** par catégorie (Protéines, Féculents, Légumes, Légumineuses, Crudités). Chaque équivalent porte : son nom (`food`), sa portion **crue/cuite** (`portionRaw` / `portionCooked`) et ses **calories**.

Les données sont stockées :

- côté serveur dans les tables `dietaryRules` et `mealPlans` (`shared/schema.ts`),
- côté téléphone via `AsyncStorage` dans `lib/nutrition-context.tsx`

(clés `nm_profile` , `nm_meals`).

3. Le plan arrive dans l'app de la cliente

La cliente ouvre l'onglet **Nutrition → Plan** (`app/(tabs)/nutrition.tsx`).

Le plan est récupéré et affiché, organisé par **repas de la journée**. Chaque

repas contient une ou plusieurs **recettes** qu'elle peut **déplier**.

4. Le détail d'une recette : ingrédients classés par famille

Les ingrédients sont rangés par **famille**, chacune avec sa couleur :

- **Protéine** (rouge)
- **Féculent** (orange)
- **Légume / Légumineuse / Crudité** (verts)

Pour chaque aliment : **nom, poids cru / cuit, calories**.

Le poids cru/cuit compte : selon la cuisson, un aliment perd ou prend de l'eau

(le riz gonfle, la viande réduit). L'app affiche les deux.

Code — couleur de famille et affichage du poids

```
const igColor = (g: string) =>
  g === 'protein' ? '#E74C3C'
  : g === 'starch' ? '#F39C12'
  : g === 'vegetable' ? '#27AE60'
  : g === 'legume' ? '#2ECC71'
  : g === 'crudite' ? '#1ABC9C'
  : Colors.muted;

const fmtWeight = (ing: any) =>
  ing.gramsRaw && ing.gramsCuit
  ? `${ing.gramsRaw}g cru / ${ing.gramsCuit}g cuit`
  : ing.quantityRaw || (ing.grams ? `${ing.grams}g` : '');
```

5. Le cœur du système : « Changer l'aliment » (équivalences)

Sous un aliment **principal**, un bouton « **Changer l'aliment** » ouvre la liste des **équivalents de la même famille**.

****Règle d'or : on ne remplace que dans la même famille, à budget calorique équivalent.**** Une protéine se remplace par une protéine, un féculent par un féculent. Les calories restent stables — **c'est la quantité qui s'ajuste**.

Exemple (famille Protéine, ~165 kcal) :

- Blanc de poulet → 150 g cru / 112 g cuit → 165 kcal
- Cabillaud (maigre) → 170 g cru / 136 g cuit
- Saumon (gras) → 130 g cru / 104 g cuit

Code — retrouver les équivalents de la bonne famille (`getEquivItems`)

```
const getEquivItems = (group: string) => {
  const norm = (s: string) =>
    s.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '').trim();
  for (const eq of equivalences) {
    const cat = norm(eq.category || '');
    if (group === 'protein' && /^prot[é]lines?/.test(cat)) return eq.items || [];
    if (group === 'starch' && /^f[é]culents?/.test(cat)) return eq.items || [];
    if (group === 'vegetable' && /^l[é]gumes?(?:s+cuits)?$/ .test(cat)) return eq.items || [];
    if (group === 'legume' && /^l[é]gumineuses?/.test(cat)) return eq.items || [];
    if (group === 'crudite' && /^crudit[é]s?/.test(cat)) return eq.items || [];
  }
  return [];
};
```

Code — la ligne d'un aliment + bouton + menu des équivalents (`renderSwapRow`)

```
const renderSwapRow = (ing: any, iIdx: number) => {
  const swapKey = `${recipeKey}_${ing.group}`; // mémorise par recette + famille
  const currentSwap = ingSwaps[swapKey]; // l'équivalent choisi (s'il existe)
  const equivItems = getEquivItems(ing.group);
  const showDD = openSwapDrop === swapKey;

  // nom / poids / calories affichés = ceux de l'équivalent choisi, sinon l'aliment d'origine
  const dName = currentSwap ? currentSwap.food : ing.name;
  const dWeight = currentSwap
    ? (currentSwap.portionRaw && currentSwap.portionCooked && currentSwap.portionCooked !== '-'
      ? `${currentSwap.portionRaw} cru / ${currentSwap.portionCooked} cuit`
      : currentSwap.portionRaw || currentSwap.portion || '')
    : fmtWeight(ing);
  const dCal = currentSwap ? currentSwap.calories : ing.calories;
  const hasDrop = equivItems.length > 0;

  // ... rendu : ligne (nom / poids / kcal), bouton "Changer l'aliment",
  // puis liste déroulante. Sélectionner un item =>
  // setIngSwaps(prev => ({ ...prev, [swapKey]: eqItem }))
  // Revenir à l'origine => on supprime la clé du swapKey.
};
```

Code — les états React qui mémorisent les choix

```
const [vegChoices, setVegChoices] = useState<Record<string, number>>({}); // quel légume (niveau 1)
const [ingSwaps, setIngSwaps] = useState<Record<string, any>>({}); // quel équivalent choisi
const [openSwapDrop, setOpenSwapDrop] = useState<string | null>(null); // quelle liste est ouverte
```

Important : choisir une équivalence **ne déclenche aucun appel serveur**. Le

choix est instantané et purement local (état React), et fonctionne hors ligne.

6. Le cas particulier des légumes : deux niveaux

- **Niveau 1 — quel légume ?** Le plan peut proposer des alternatives directes

(ex. Courgettes / Lentilles / Tomates). La cliente coche celui qu'elle veut

(`vegChoices`).

- **Niveau 2 — l'équivalence** : une fois le légume choisi, elle peut encore le

changer dans sa propre famille avec le même bouton « Changer l'aliment ».

Si elle choisit une **légumineuse**, l'app signale qu'elle apporte **plus de fibres**.

Code — sélection du légume (niveau 1)

```
const hasVegAlts = recipe.ingredients.some((ig: any) =>
  (ig.group === 'vegetable' || ig.group === 'legume' || ig.group === 'crudite')
  && ig.alternatives?.length > 0);
const vegIng = hasVegAlts
  ? recipe.ingredients.find((ig: any) =>
    (ig.group === 'vegetable' || ig.group === 'legume' || ig.group === 'crudite')
    && ig.alternatives?.length > 0)
  : null;
const vegOptions = vegIng ? [vegIng, ...(vegIng.alternatives || [])] : [];
const selectedVegIdx = vegChoices[recipeKey] ?? 0;
```

```
const selectedVeg = vegOptions[selectedVegIdx] || vegIng;  
// onPress d'une option : setVegChoices(prev => ({ ...prev, [recipeKey]: oIdx })))
```

7. Le garde-fou avant de valider

Un **bandeau de rappel** s'affiche au-dessus des aliments modifiables : il demande de choisir **ce qu'on va RÉELLEMENT manger** avant d'ajouter au journal, pour que le journal reflète la vraie assiette.

8. Le passage au journal alimentaire

Une fois les choix faits, la cliente **ajoute le repas au journal**. Les calories et macros enregistrées sont celles de **ses choix réels** (équivalences comprises), pas de la recette d'origine. Le suivi de la journée se met à jour.

9. Catalogue d'équivalences de secours — `lib/equivalences-data.ts`

Les équivalents affichés dans le menu viennent **du plan extrait du PDF** (`clientMealPlan.equivalences`). Un **catalogue interne de secours** existe et n'est utilisé que si le plan n'en fournit pas. Il porte, par aliment : calories pour 100 g, ratio cru→cuit et portion recommandée.

Code — structure et extrait du catalogue

```
export interface FoodEquivalent {  
  name: string;  
  calPer100g: number;  
  rawToCookedRatio: number;  
  recommendedPortionRaw: number;  
  recommendedPortionCooked: number;  
  caloriesPerPortion: number;  
}  
  
export const EQUIVALENCES: EquivalenceCategory[] = [  
  {  
    title: 'Proteins',  
    items: [  
      { name: 'Chicken Breast', calPer100g: 110, rawToCookedRatio: 0.75, recommendedPortionRaw: 150,  
recommendedPortionCooked: 112, caloriesPerPortion: 165 },  
      { name: 'Salmon', calPer100g: 208, rawToCookedRatio: 0.80, recommendedPortionRaw: 130,  
recommendedPortionCooked: 104, caloriesPerPortion: 270 },  
      { name: 'Cod', calPer100g: 82, rawToCookedRatio: 0.80, recommendedPortionRaw: 170,  
recommendedPortionCooked: 136, caloriesPerPortion: 139 },  
      { name: 'Lentils', calPer100g: 116, rawToCookedRatio: 2.5, recommendedPortionRaw: 60,  
recommendedPortionCooked: 150, caloriesPerPortion: 174 },  
      // ... (poulet, dinde, thon, crevettes, œuf, tofu, bœuf 5%, porc, yaourt 0%)  
    ],  
  },  
  {  
    title: 'Starches',  
    items: [  
      { name: 'Basmati Rice', calPer100g: 350, rawToCookedRatio: 2.5, recommendedPortionRaw: 60, recommendedPortionCooked:  
150, caloriesPerPortion: 210 },  
      { name: 'Quinoa', calPer100g: 368, rawToCookedRatio: 2.5, recommendedPortionRaw: 60, recommendedPortionCooked:  
150, caloriesPerPortion: 220 },  
    ],  
  },  
];
```

```

    { name: 'Sweet Potato', calPer100g: 86, rawToCookedRatio: 0.85, recommendedPortionRaw: 200, recommendedPortionCooked: 170, caloriesPerPortion: 172 },
    // ... (riz complet, pâtes complètes, pomme de terre, boulgour, semoule, pain complet, flocons d'avoine)
  ],
},
{
  title: 'Vegetables',
  items: [
    { name: 'Broccoli', calPer100g: 34, rawToCookedRatio: 0.90, recommendedPortionRaw: 200, recommendedPortionCooked: 180, caloriesPerPortion: 68 },
    { name: 'Zucchini', calPer100g: 17, rawToCookedRatio: 0.85, recommendedPortionRaw: 200, recommendedPortionCooked: 170, caloriesPerPortion: 34 },
    { name: 'Spinach', calPer100g: 23, rawToCookedRatio: 0.30, recommendedPortionRaw: 200, recommendedPortionCooked: 60, caloriesPerPortion: 46 },
    // ... (haricots verts, poivron, tomate, champignons, asperges)
  ],
},
];

```

Code — proposer des substitutions à partir d'une liste d'ingrédients

```

export function getSubstitutions(ingredients: string[]): RecipeSubstitution[] {
  const subs: RecipeSubstitution[] = [];
  const proteinKeywords = ['chicken', 'turkey', 'salmon', 'tuna', 'shrimp', 'egg', 'tofu', 'beef', 'pork', 'cod'];
  const starchKeywords = ['rice', 'quinoa', 'pasta', 'sweet potato', 'potato', 'bulgur', 'couscous', 'oats'];
  const veggieKeywords = ['broccoli', 'green beans', 'zucchini', 'spinach', 'bell pepper', 'tomato', 'mushrooms', 'asparagus'];

  for (const ing of ingredients) {
    const lower = ing.toLowerCase();
    // protéine : on propose d'autres protéines (sauf la même), avec portions crue/cuite
    for (const keyword of proteinKeywords) {
      if (lower.includes(keyword)) {
        const others = EQUIVALENCES[0].items
          .filter(item => !item.name.toLowerCase().includes(keyword))
          .slice(0, 4);
        if (others.length > 0) {
          subs.push({
            original: ing,
            alternatives: others.map(alt => ({
              name: alt.name,
              portionRaw: `${alt.recommendedPortionRaw}g raw`,
              portionCooked: `${alt.recommendedPortionCooked}g cooked`,
            })),
          });
        }
        break;
      }
    }
    // ... même logique pour féculents et légumes
  }
  return subs;
}

```

En résumé

- Le coach importe le PDF → 2. l'IA en extrait règles, calories, recettes et **tables d'équivalences** → 3. le plan apparaît dans l'app → 4. les ingrédients sont rangés par famille avec poids cru/cuit + kcal → 5. la cliente peut

changer un aliment par un équivalent de la même famille (calories stables, portion ajustée) → 6. choix de légume à deux niveaux → 7. rappel de choisir ce qu'on mange vraiment → 8. ajout au journal avec les vraies valeurs.

Tout le remplacement est **local et instantané** ; rien n'est inventé : les équivalents viennent du plan, avec un catalogue interne en secours.